# Инкапсуляция

**Инкапсуляция** – данные и методы которые существуют внутри класса, отделенные от интерфейса, объединенные в единое целое.[1]

**Инкапсуляция** – Один объект будет содержать как свои данные, так и поведения, и сможет скрыть то, что ему потребуется, от других объектов.[2]

**Уровни:**
- `private` - доступны исключительно классу
- `protect` - доступны наследникам, но не доступно извне
- `public` - доступны извне

## Сокрытие

**Сокрытие** – защищенные данные от изменений извне.[1]

При проектировании языка python исходили из концепции: “все мы взрослые люди”. Поэтому в python нет классических `public`, `protect` и `private` полей.

Однако вместо этого есть соглашение о наименовании доступов. И PEP 8 они описываются так:

- `_single_leading_underscore`: слабый индикатор “внутреннего использования”. Например, `from M import *` не импортирует объекты, имена которых начинаются с подчеркивания.
- `single_trailing_underscore_`: используется по соглашению во избежание конфликтов с ключевым словом Python, например:
- `tkinter.Toplevel(master, class_='ClassName')`
- `__double_leading_underscore`: при именовании атрибута класса вызывает искажение имени (внутри класса FooBar `__boo` становится `_FooBar__boo`; см. ниже).
- `__double_leading_and_trailing_underscore__`: «магические» объекты или атрибуты, которые находятся в управляемых пользователем пространствах имен. Например, `__init__`, `__import__` или `__file__`. Никогда не придумывайте такие имена; используйте их только в соответствии с документацией.

[1] определение из лекции по ООП от 10.10.26
[2] Объектно-ориентированное мышление – Мэтт Вайсфельд